INF201 - Deliverable 1: Kachina Aschenberg Øwre & Seline Heggelund Knutsen

Link to our GitHub repository: https://github.com/kacoew/INF201-CAs

Task 1:

In [1]:
```python
# Imported libraries
import requests # HTTP library for making requests to the Frost API
import pandas as pd # Organizes and analyzes the weather data
import yaml # Structures data into a .yml file for easy reading and
sharing
import matplotlib.pyplot as plt # Visualizes the data in a graph
import matplotlib.dates as mdates  # Formats dates on the x-axis

# Read the client ID from a text file outside the repository
with open('/Users/kachinaaschenberg/Documents/NMBU/INF201/Frost_API.txt',
'r') as infile:
    client_id = infile.read().strip()


def frost_request(client_id: str, sources: str, elements: str,
referencetime: str) -> pd.DataFrame | None:
    '''
        Gets data from the Frost API and returns it as a pandas DataFrame
        (based on MET's Python example script (MET Norway, n.d.))
        Args:
            client_id (str): Client ID for the Frost API.
            sources (str): Station ID, e.g., 'SN17850'.
            elements (str): Element, e.g., 'mean(air_temperature P1D)'
            referencetime (str): Time range, e.g., '2025-01-01/2025-12-
31'.
        Returns:
            pd.DataFrame | None: DataFrame with the columns 'date' and
'value',
            or None if the request failed.
        '''

    # Define the endpoint and parameters for the API request
    endpoint = 'https://frost.met.no/observations/v0.jsonld'
    parameters = {
        'sources': sources,
        'elements': elements,
        'referencetime': referencetime,
        'timeoffsets': 'default',
    }

    # HTTP GET request to Frost API with authentication using the client
ID
    r = requests.get(endpoint, params=parameters, auth=(client_id, ''))

    # Check if the request was successful (status code 200)
    if r.status_code != 200:
        print(f'Error {r.status_code}: {r.json()}')
        return None

    data = r.json()
```

```
    if 'data' not in data:
        print('No data received')
        return None

    # Check if the response contains data
    records = [] # List to store the records from the API response
    for item in data['data']: # Iterate through each item in the 'data'
field of the JSON response
        date = item['referenceTime']
        value = item['observations'][0]['value']
        records.append((date, value))



    df = pd.DataFrame(records, columns=['date', 'value']) # Create a
pandas df from the records list
    df['date'] = pd.to_datetime(df['date']) # Convert the 'date' column
to datetime format for an easier analysis

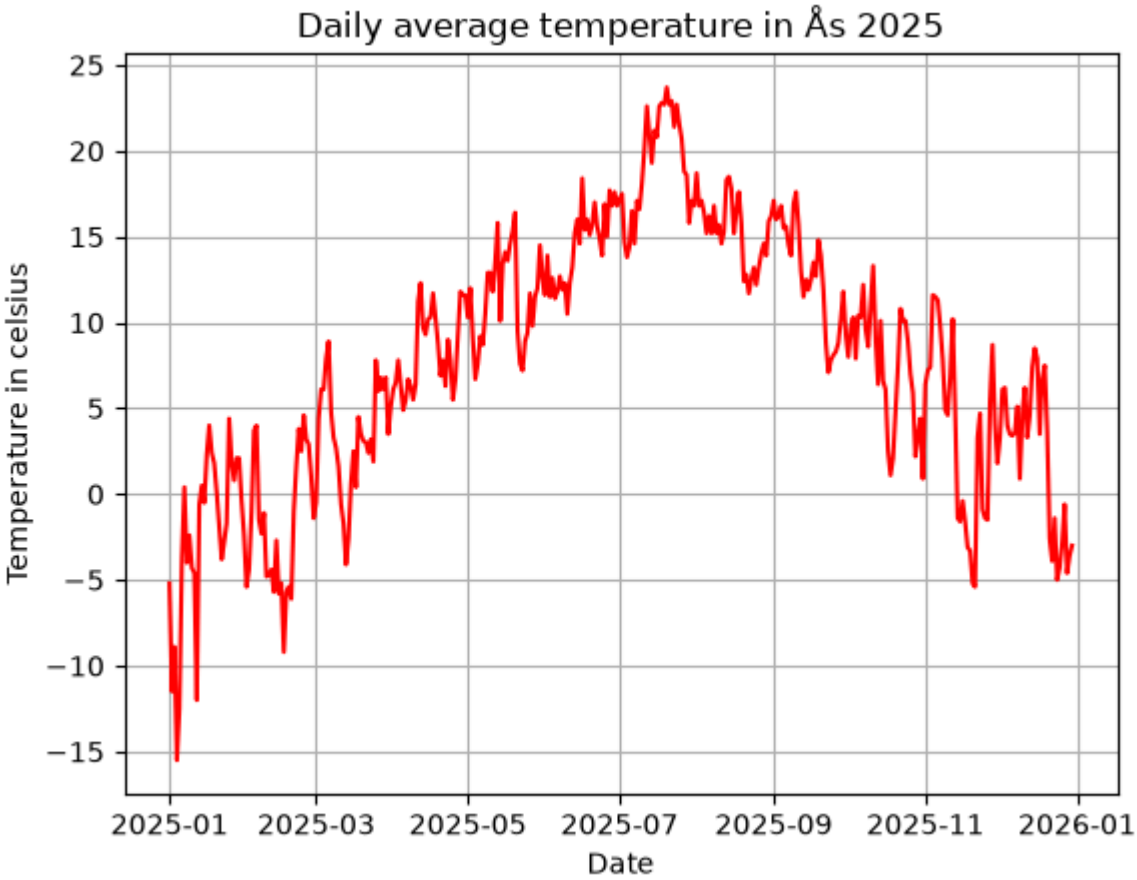
    return df


# Get the data for 2025 from the Frost API
df = frost_request(
    client_id=client_id,
    sources='SN17850',
    elements='mean(air_temperature P1D)',
    referencetime='2025-01-01/2025-12-31'
)

# Plot the data
plt.plot(df['date'], df['value'], color='red')
plt.title('Daily average temperature in Ås 2025')
plt.xlabel('Date')
plt.ylabel('Temperature in celsius')
plt.grid(True)
plt.show()

# Make a summary table
summary = {
    'Mean': df['value'].mean(),
    'Median': df['value'].median(),
    'Min': df['value'].min(),
    'Max': df['value'].max(),
}

summary_df = pd.DataFrame(summary, index=['2025']).round(1)
print(summary_df)
```



	Mean	Median	Min	Max
2025	7.9	8.4	−15.5	23.7

Discussion of Task 1

The plot shows the temperature for Ås in 2025. The mean temperature for 2025 was 7.9 °C and the median was 8.4 °C.

- The coldest day (−15.5 °C) was in January, and the warmest day (23.7 °C) was in July, at the same time as the heatwave we found in Task 4.
- The median is a bit higher than the mean. A few very cold winter days pull the mean down more than the warm summer days pull it up.
- The temperature changes much more from day to day in winter than in summer. In early January it went from about −15 °C to around 0 °C in less than a week.
- These are daily *mean* temperatures, so the coldest and warmest hours of the year were more extreme than the values in the table.

Note: We used `referencetime = '2025-01-01/2025-12-31'` as given in the task. The end date is not included, so 31 December is missing. This is why the 2025 median is 8.3 °C in the table in Task 3, where we get the whole year.

Task 2:

In [2]:
```
df_temp = frost_request(
    client_id=client_id,
    sources="SN17850",
    elements="mean(air_temperature P1D)",
    referencetime="2025-01-01/2025-12-31"
```

```
)

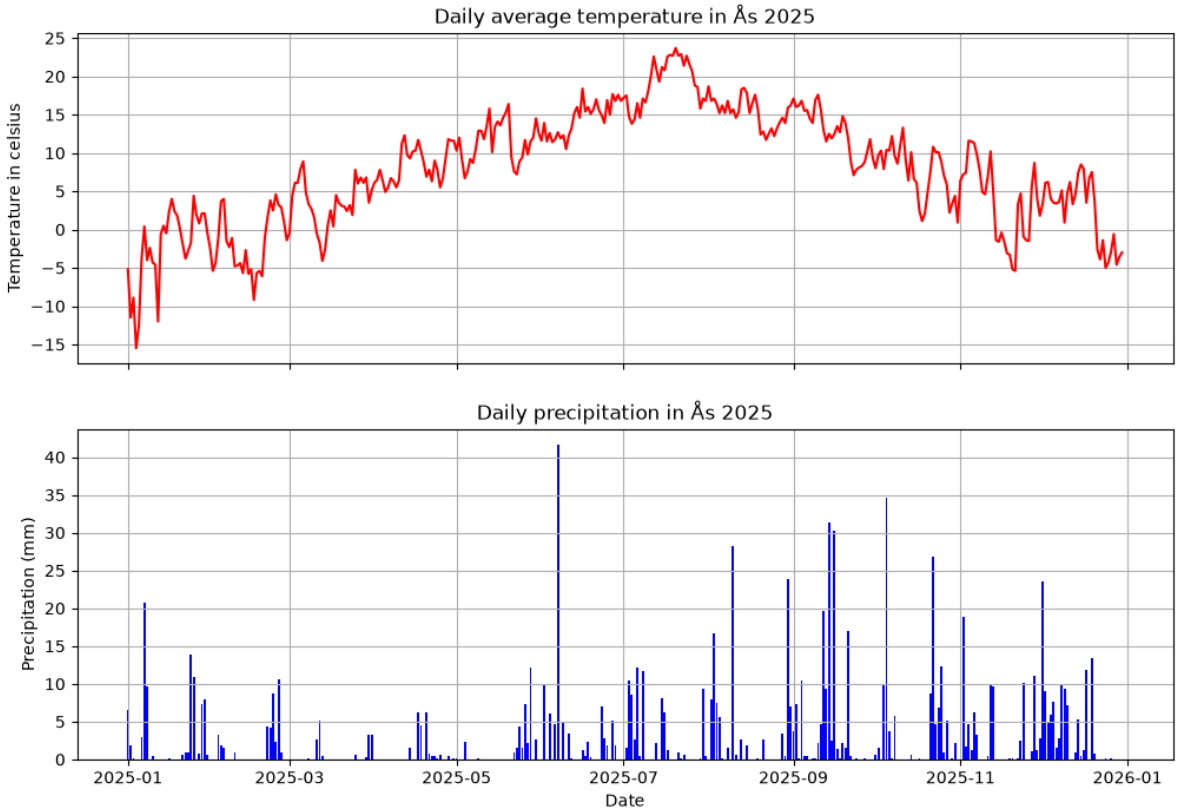
df_rain = frost_request(
    client_id=client_id,
    sources="SN17850",
    elements="sum(precipitation_amount P1D)",
    referencetime="2025-01-01/2025-12-31"
)

fig, ax = plt.subplots(2, 1, figsize=(12, 8), sharex=True)

# Temperature
ax[0].plot(df_temp['date'], df_temp['value'], color='red')
ax[0].set_title('Daily average temperature in Ås 2025')
ax[0].set_ylabel('Temperature in celsius')
ax[0].grid(True)

# Precipitation
ax[1].bar(df_rain['date'], df_rain['value'], color='blue')
ax[1].set_title('Daily precipitation in Ås 2025')
ax[1].set_ylabel('Precipitation (mm)')
ax[1].grid(True)

plt.xlabel('Date')
plt.show()
```



Discussion of Task 2

We put temperature and precipitation in two subplots on top of each other with a shared x-axis.

- **Temperature** changes gradually from day to day, so it's shown as a line.

- **Precipitation** is shown as bars, because each value is the total amount for one day and many days have no precipitation at all. A line would make it look like it rained between the measurements.

What we learn from the plots:

- March to May was dry, with only a handful of days with more than 5 mm.
- The largest amount in one day was about 42 mm, in early June.
- August to December was the wettest part of the year. It had more days with precipitation and several days with 25–35 mm.
- In January and February the precipitation probably fell as snow, because the temperature was often below zero.
- We don't see a clear connection between temperature and precipitation from day to day. Each of them mostly follows its own seasonal pattern.

Task 3:

In [3]:
```python
# Imported libraries
import matplotlib.dates as mdates  # Formats dates on the x-axis


def get_mean_temperature(client_id: str, station_id: str, year: int) ->
pd.DataFrame | None:
    '''
    Helper function that calls frost_request with the right parameters.
    Gets the daily mean air temperature for one station a whole year.

    Args:
        client_id (str): Personal client ID for the Frost API.
        station_id (str): Station ID, e.g. 'SN17850'.
        year (int): Year to get data from, e.g. 1925.

    Returns:
        pd.DataFrame | None: DataFrame with the columns 'date' and
'value',
        or None if the request failed.
    '''
    return frost_request(client_id, station_id, 'mean(air_temperature
P1D)',
                         f'{year}-01-01/{year + 1}-01-01')


def summary_table(data: dict[str, pd.DataFrame]) -> pd.DataFrame:
    '''
    Makes a summary table with mean, median, min and max temperature for
each year in the dictionary.

    Args:
        data (dict[str, pd.DataFrame]): Maps a label like '1925' to a
DataFrame
            with a 'value' column.

    Returns:
        pd.DataFrame: One row per label and one column per statistic,
        rounded to one decimal.
```

```
    '''

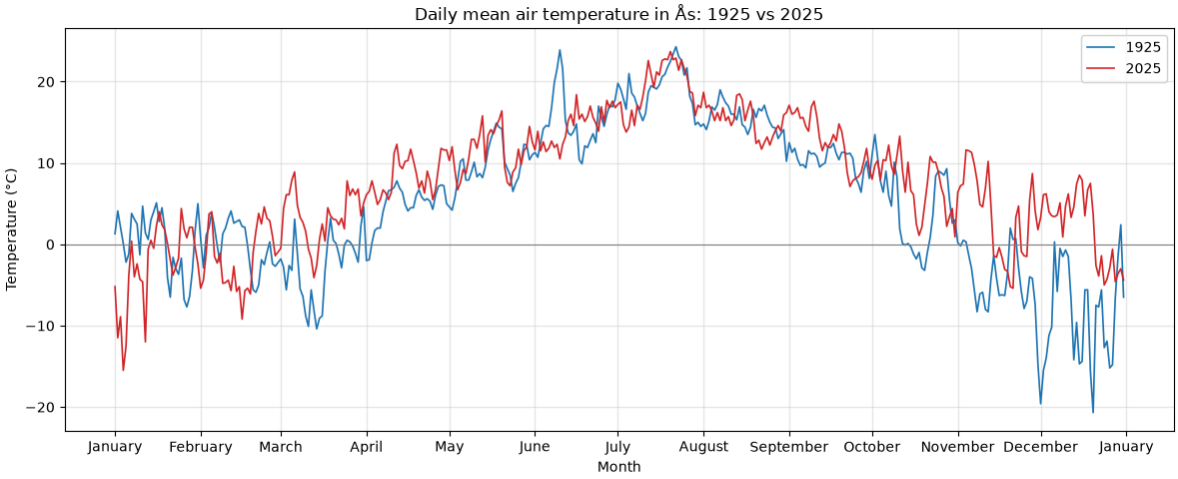
    # Create a dictionary to hold the summary statistics for each label
    rows = {}
    for label, df in data.items(): # Iterate through each label and its
corresponding df in the dictionary
        rows[label] = {
            'Mean': df['value'].mean(),
            'Median': df['value'].median(),
            'Min': df['value'].min(),
            'Max': df['value'].max(),
        }
    return pd.DataFrame(rows).T.round(1)


# Get the data for both years
df_1925 = get_mean_temperature(client_id, 'SN17850', 1925)
df_2025 = get_mean_temperature(client_id, 'SN17850', 2025)

# Plot both years on the same calendar axis (month), so they can be
compared
fig, ax = plt.subplots(figsize=(12, 5)) # Gives a figure and axis for the
plot with a set size
ax.plot(df_1925['date'].apply(lambda d: d.replace(year=2000)),
df_1925['value'], color='tab:blue', label='1925', linewidth=1.2) # Plot
the temperature values for each year with  color and label
ax.plot(df_2025['date'].apply(lambda d: d.replace(year=2000)),
df_2025['value'], color='tab:red', label='2025', linewidth=1.2) # Plot
the temperature values for each year with color and label


ax.xaxis.set_major_locator(mdates.MonthLocator()) # Set the x-axis to
each month
ax.xaxis.set_major_formatter(mdates.DateFormatter('%B')) # Format the x-
axis labels to show the whole month name
ax.axhline(0, color='grey', linewidth=0.8)  # line at 0 °C
ax.set_title('Daily mean air temperature in Ås: 1925 vs 2025') # Set the
title of the plot
ax.set_xlabel('Month') # Set the label for the x-axis
ax.set_ylabel('Temperature (°C)') # Set the label for the y-axis
ax.grid(True, alpha=0.3) # Enable grid lines on the plot with a specified
transparency
ax.legend() # Add a legend to the plot to identify the lines for each
year
plt.tight_layout() # Prevent overlapping elements
plt.show() # Display the plot

# Summary table for both years
print(summary_table({'1925': df_1925, '2025': df_2025}))
```



	Mean	Median	Min	Max
1925	5.3	5.3	−20.7	24.3
2025	7.9	8.3	−15.5	23.7

Discussion of Task 3

We compared the two years on the same calendar axis. We set the year of all dates to 2000 (a leap year), so only the month and day matter.

2025 was clearly warmer than 1925, with a mean of 7.9 °C in 2025 and 5.3 °C in 1925, a difference of 2.6 °C, and the median was 3.0 °C higher. Most of the difference comes from the cold part of the year:

- The coldest day in 1925 was −20.7 °C and −15.5 °C in 2025. The warmest days were almost the same (24.3 °C and 23.7 °C).
- December 1925 had several periods below −10 °C, but in 2025 November and December mostly stayed between about −5 and +10 °C.
- March and April were also warmer in 2025.
- The summers were similar, but 1925 had a warm period in early June that was much warmer than 2025 at the same time. The start of January was also milder in 1925 than in 2025.

This fits with the warming observed in Norway over the last 100 years, but this is data from only two years. The weather varies a lot from year to year and climate is usually compared using averages over decades. The measurements from 1925 may also have been made with different instruments and methods than today.

Task 4:

In [4]:
```
def find_heatwaves(df: pd.DataFrame, threshold: float = 27.0, min_days:
int = 5) -> list:
    '''
    Finds periods where the maximum temperature is >= threshold for at
least min_days consecutive days
    Args:
        df (pd.DataFrame): DataFrame containing the temperature data with
'date' and 'value' columns.
        threshold (float): Temperature threshold to define a heatwave
(default is 27.0 degrees).
        min_days (int): Minimum number of consecutive days to define a
```

```
heatwave (default is 5 days).
    Returns:
        list: List of (start_date, end_date) for each heatwave
    '''

    # Filter the df to only include days temperature is above the
threshold
    hot = df[df['value'] >= threshold].sort_values('date')

    # Hot days get put into periods of consecutive days (group 1, group
2, etc.)
    new_period = hot['date'].diff() != pd.Timedelta(days=1) # New period
starts when its more than one day since the previous hot day
    period_id = new_period.cumsum() # assigns group numbers to each
period of consecutive hot days


    # Create a list of heatwaves that are at least 5 days long
    heatwaves = []
    for _, period in hot.groupby(period_id): # Iterate through each group
of consecutive hot days
        if len(period) >= min_days: # If the period is at least 5 days
long, add it to the heatwaves list
            heatwaves.append((period['date'].iloc[0],
period['date'].iloc[-1])) # Append a tuple of (start_date, end_date) to
the heatwaves list
    return heatwaves

def count_heatwaves(client_id: str, station_id: str, station_name: str,
year: int, filename: str = None) -> int:
    '''
    Counts the number of heatwaves for a station in a given year and
writes a summary to a YAML file
    Args:
        client_id (str): Client ID for the Frost API.
        station_id (str): Station ID, e.g., 'SN17850'.
        station_name (str): Name of the station, e.g., 'Ås'.
        year (int): Year for which to count heatwaves, e.g., 2025
        filename (str): Optional filename for the YAML summary file. If
None, a default name
    Returns:
        int: Number of heatwaves found for the specified station and
year.
    '''

    # Get the maximum temperature data for the specified station and year
    df_max = frost_request(client_id, station_id, 'max(air_temperature
P1D)',
                           f'{year}-01-01/{year}-12-31')
    if df_max is None:
        return 0

    # Find heatwaves in the data
    heatwaves = find_heatwaves(df_max)

    # Write a summary of the heatwaves to a YAML file, including set
parameters and the number of heatwaves found
    summary = {
        'dates': [f'{start:%Y-%m-%d} - {end:%Y-%m-%d}' for start, end in
```

```
heatwaves],
        'station_id': station_id,
        'station_name': station_name,
        'year': year,
    }
    # Creates a default filename based on the station ID and year if no
name is provided
    if filename is None:
        filename = f'heatwaves_{station_id}_{year}.yml'

    # Write the summary to a YAML file
    with open(filename, 'w') as outfile:
        yaml.dump(summary, outfile)

    # Return the number of heatwaves found
    return len(heatwaves)

# Example usage of the count_heatwaves function
n = count_heatwaves(client_id, 'SN17850', 'Ås', 2025)
print(f'Number of heatwaves in Ås in 2025: {n}')
```

```
Number of heatwaves in Ås in 2025: 1
```

Discussion of Task 4

In Norway a heatwave is defined as at least five days in a row where the maximum temperature is 27 °C or higher. That is why we used `max(air_temperature P1D)` here, and not the daily mean as in the other tasks.

We split the work into two functions:

- `find_heatwaves()` : keeps only the days where the maximum temperature is at least 27 °C. Then it groups days that come right after each other: a new group starts every time there is more than one day since the previous hot day (using `diff()` and `cumsum()`). Groups with at least five days are counted as heatwaves. The threshold and the minimum number of days are parameters with 27 °C and 5 days as default values, so the function can also be used with other definitions.
- `count_heatwaves()` gets the data for any station and year, uses `find_heatwaves()` , writes a summary to a YAML file in the same format as the example file and returns the number of heatwaves. The file name is made from the station ID and the year, for example `heatwaves_SN17850_2025.yml` .

In Ås there was 1 heatwave in 2025, from 16 to 22 July (7 days). This is the same warm period we see in the temperature plots in Task 1 and Task 5, where the daily mean temperature reached 23.7 °C.

Task 5:

In [5]:
```
def moving_average(df: pd.DataFrame, window: int = 7) -> pd.DataFrame:
    '''
    Calculates a moving average of the temperature over a specified
number of days.
    Args:
```

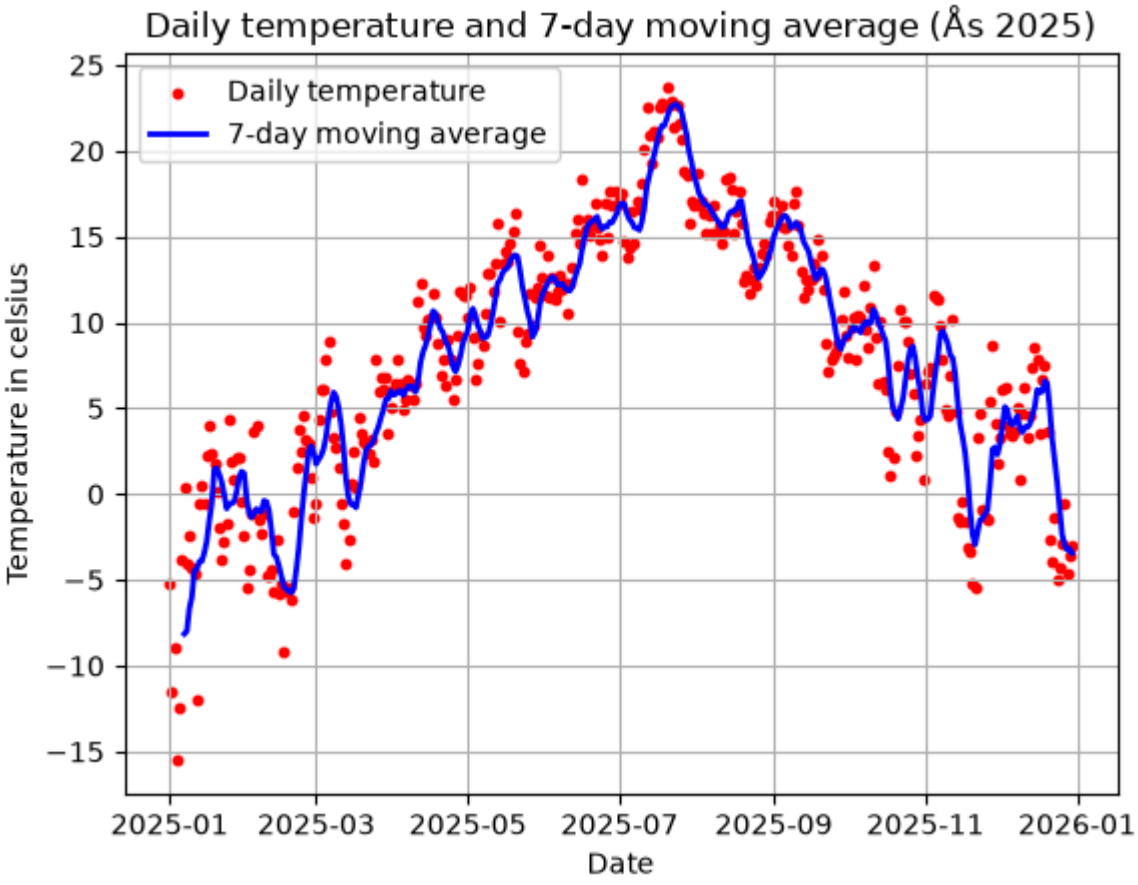
```
        df (pd.DataFrame): DataFrame containing 'date' and 'value'
columns.
        window (int): Number of days to include in the moving average.
    Returns:
        pd.DataFrame: A copy of the DataFrame with an additional
'moving_avg' column
    '''
    df_ma = df.copy()
    df_ma['moving_avg'] = df_ma['value'].rolling(window=window).mean()
    return df_ma


df_temp_ma = moving_average(df_temp, window=7)


# Daily temperature as scatter plot
plt.scatter(df_temp_ma['date'], df_temp_ma['value'], color='red', s=10,
label='Daily temperature')

# Moving average as line
plt.plot(df_temp_ma['date'], df_temp_ma['moving_avg'], color='blue',
linewidth=2, label='7-day moving average')

plt.title('Daily temperature and 7-day moving average (Ås 2025)')
plt.xlabel('Date')
plt.ylabel('Temperature in celsius')
plt.legend()
plt.grid(True)
plt.show()
```



Discussion of Task 5

The 7-day moving average is calculated with `rolling(window=7).mean()`, each value is the mean of that day and the six days before. The first six days don't have a full window, so the line starts on 7 January.

- The moving average removes most of the day-to-day variation. That makes the seasonal trend and the longer warm and cold periods easier to see, for example the cold spells in early January and in February, the warm period in July (the heatwave from Task 4) and the cold period at the end of November.
- The extreme values are smoothed out. The coldest day was –15.5 °C, but the lowest value of the moving average is only about –8 °C.
- The points are more spread out around the line in winter than in summer. This matches what we saw in Task 1: the weather is more variable in winter.
- Because the window only looks backwards, the line lags a few days behind the actual temperature. With `rolling(window=7, center=True)` the window would be centred on each day, and the line would follow the points more closely in time.

Sources

MET Norway. (n.d.). *Python example script*. Frost. https://frost.met.no/python_example.html